Solver and Financial API Documentation

American Forests Carbon Dashboard

Spatial Informatics Group

October 2026

Purpose

This document describes the financial (proforma) and solver functionality of the carbon model service. It covers how carbon outputs become credits, how the yearly proforma and summary metrics are computed, the scenario and solver endpoints, the Solver page of the dashboard, and the Python client. It complements the Model Service Documentation, which covers the carbon models, registry, and geo endpoints.

Current Directory Context

Important paths:

- model_service/model.py: compute_carbon_units, compute_proforma, compute_summaries, default_scenario, run_scenario, and the closed-form acreage solver.
- model_service/tpa_sweep.py: trees-per-acre (TPA) sweep and range solver.
- model_service/schemas.py: ProformaRequest, ScenarioRequest, SolveDirective, TpaSweepRequest, and their responses.
- model_service/main.py: the /proforma, /scenario endpoints and bulk evaluation.
- conf/base/proforma_presets.json: default financial assumptions and per-protocol overrides.
- conf/base/protocol_rules.json: per-protocol carbon-unit rules (coefficient, buffer).
- utils/functions/solver.py: the dashboard Solver page and the payload builders it uses.
- utils/functions/plant_design.py: the Planting Design page, including the Project Financials section.
- aff_dash_client/: Python client and CLI for scenario runs and solves.

Pipeline Overview

Every financial result follows the same chain:

1. Carbon projection: FVS-derived models (or the coefficient fallback) produce annual on-site carbon (ABLD_C) for a variant, location, PCT level, survival, site index, and species mix.
2. Carbon units: compute_carbon_units converts carbon into creditable units per protocol, applying the protocol's coefficient and buffer deduction.
3. Proforma: compute_proforma turns per-acre credits into a yearly cash-flow table for a given acreage and financial assumptions.
4. Summaries: compute_summaries reduces the proforma to Total Net Revenue, NPV at a chosen horizon, and NPV per acre.
5. Solvers: the acreage solver inverts step 3 and 4 for net_acres; the TPA sweep repeats steps 1 to 4 across planting densities.

The scenario endpoints run this whole chain in one call. The /proforma/compute endpoint runs only steps 3 and 4 on credit rows the caller already has.

Carbon Units

Compute_carbon_units(df_carbon, protocols, protocol_rules) takes rows of Year and ABLD_C and returns rows of Year, CU, and Protocol.

For each protocol:

- On-site CO2e is ABLD_C times 3.667 times the protocol coefficient.
- The series is linearly interpolated to every year between the first and last modeled year.
- The baseline is zero, so the annual creditable change is the year-over-year difference of the project series. Because of the difference, the first modeled year has no CU row.
- If the protocol applies a buffer, BUF equals the annual change times the protocol's buffer fraction and is deducted. CU is the annual change minus BUF.

Protocol rules come from protocol_rules.json. Each protocol has coeff, BUF, and apply_buf. If a protocol has no rule, the service falls back to ACR, then CAR, then VERRA. Supported protocols are ACR, CAR, VERRA, GS, and ISO.

Current configured protocol rules:

Protocol	Coefficient	Buffer
ACR	1	20%
CAR	1	20%
VERRA	1	20%
GS	1	No buffer
ISO	1	25%

Current scope note: the implementation explicitly represents protocol coefficients and buffer-pool deductions. It does not separately model leakage deductions, uncertainty deductions, or protocol-specific crediting-period rules unless those effects are represented indirectly through the configured protocol coefficient or buffer assumptions. The current baseline treatment is zero baseline, and creditable units are derived from year-over-year increases in modeled project CO2e.

CU values are per acre. The proforma multiplies them by net acres.

Financial Inputs

Financial assumptions are supplied per protocol as financial_params. Defaults come from proforma_presets.json; the protocol_overrides block in that file supplies protocol-specific values for the fee fields. The API fills in any field the caller leaves out.

Field	Meaning	Unit	Default
price_per_ert_initial	Starting market price per credit	$ per credit	25.00
credit_price_increase	Annual increase in credit price	percent per year	2.0
discount_rate	Rate for discounting cash flows in NPV	percent per year	6.0
anticipated_inflation	Annual inflation applied to survey costs and added to the discount rate	percent per year	2.0
planting_cost	Upfront planting cost, including site preparation and labor	$ per acre, once	1000
num_plots	Number of Continuous Forest Inventory (CFI) sampling plots	count	250

Field	Meaning	Unit	Default
cost_per_cfi_plot	Cost to establish and maintain one CFI plot	$ per plot	150
registry_fees	Annual registry registration and maintenance fee	$ per year	500
issuance_fee_per_ert	Registry fee charged on each credit issued	$ per credit	0.15
validation_cost	One-time third-party validation	$, once	45000
verification_cost	Recurring independent verification	$, every 5 years	25000

Percent handling. The three rate fields are percent-shaped in the UI (6.0 means 6 percent). The service accepts either form: any rate value greater than 1 is divided by 100, and values of 1 or less are treated as fractions already. The dashboard converts to fractions before calling the API. When reading the echoed inputs from a response, rates are always fractions.

Protocol overrides. The current presets carry per-protocol values for cost_per_cfi_plot, registry_fees, issuance_fee_per_ert, validation_cost, verification_cost, and planting_cost. For example, ACR's registry fee is 8500 and VERRA's is 11750. GET /scenario/defaults returns ACR's merged values; the dashboard applies the selected protocol's overrides the same way.

Current protocol financial overrides are:

Protocol	Registry Fees	Issuance Fee per ERT	Validation Cost	Verification Cost	Cost per CFI Plot	Planting Cost (per acre)
ACR	$8,500	$0.20	$15,000	$50,000	$150	$1,000
CAR	$1,500	$0.20	$15,000	$50,000	$150	$1,000
VERRA	$11,750	$0.23	$15,000	$50,000	$150	$1,000
GS	$7,500	$0.25	$15,000	$50,000	$150	$1,000
ISO	$0	$0	$0	$0	$150	$1,000

Proforma Calculation

compute_proforma(df_ert_ac, params) takes CU rows (Year, CU, Protocol) and a params dict holding the financial fields plus net_acres and year_start. It returns one row per protocol and year with these columns:

- Project_acres: net_acres.
- CU_ac and CU: credits per acre, and credits for the whole project.
- CUs_Sold: credits are sold in five-year blocks. In year_start and every fifth year after it, the row sells the sum of the previous five years' CU. All other years sell nothing.
- CU_Credit_Price: price_per_ert_initial grown by credit_price_increase each year since year_start.
- Total_Revenue: CUs_Sold times CU_Credit_Price.
- Validation_and_Verification: validation_cost once in the upfront year, plus verification_cost in each sale year after the start.

- Survey_Cost: num_plots times cost_per_cfi_plot times (1 plus anticipated_inflation), charged in the year before each sale year.
- Registry_Fees: registry_fees every year.
- Issuance_Fees: CUs_Sold times issuance_fee_per_ert.
- Planting_Cost: planting_cost times net_acres, once in the upfront year.
- Total_Costs: the sum of the five cost columns.
- Net_Revenue: Total_Revenue minus Total_Costs.

The upfront year is the first proforma row at or after year_start. Because the carbon-unit difference removes the first modeled year, upfront costs land on the earliest surviving year, which keeps them undiscounted in NPV.

year_start is the first year present in the CU rows. The modeled horizon is 35 years after it (PROFORMA_YEARS_ADVANCE).

Summary Metrics

compute_summaries(df_pf, params, npv_years) returns one row per protocol:

- total_net: the sum of Net_Revenue over every proforma year. This is Total Net Revenue (TNR) and is not discounted.
- npv_yr: the net present value of Net_Revenue for years up to year_start plus npv_years. The discount rate is anticipated_inflation plus discount_rate, as fractions. The first row is period zero.
- npv_year: the horizon used.
- npv_per_acre: npv_yr divided by net_acres.

The dashboard's cumulative net revenue chart uses the same discount definition, so the discounted curve's value at the horizon equals npv_yr, and the year it crosses zero is the discounted break-even year.

Financial and Scenario Endpoints

All endpoints except GET /health require an Authorization: Bearer header. See the Authentication section of the Model Service Documentation.

GET /proforma/presets

Returns proforma_presets.json as-is: the default financial fields plus the protocol_overrides block.

POST /proforma/compute

Runs the proforma and summaries on credit rows the caller already has. Use this when carbon units were computed separately, for example from POST /carbon/units.

Request (ProformaRequest):

- df_ert_ac: list of rows with Year, CU, Protocol.
- params: the financial fields plus net_acres and year_start. npv_year is optional and defaults to 20 for this endpoint.

Response (ProformaResponse):

- proforma_rows: the full yearly table described above.
- summaries: one ProformaSummary per protocol with Protocol, total_net, npv_yr, npv_year, npv_per_acre.

This endpoint does not fill in missing financial fields or convert percents. Send complete params with rates as fractions.

GET /scenario/defaults

Query parameters: variant, loccode.

Returns ScenarioDefaults, a complete, ready-to-run scenario for the location: the resolved variant, survival, si, species_tpa, species_codes, pct_level (PCT0), net_acres (1000), protocols (["ACR"]), financial_params for ACR with protocol overrides applied, and npv_year (40). The variant is resolved against the live registry, so a base code such as PN or a sub-variant such as WC_2 both work. Returns 404 if the pair has no registered model.

POST /scenario/run

Runs the full pipeline in one call and optionally solves for acreage.

Request (ScenarioRequest):

- variant, loccode: required.
- survival, si, species_tpa, pct_level, net_acres, protocols, npv_year: optional. Any value left null is filled from the location's defaults.
- financial_params: optional map of protocol to partial overrides. Missing fields fall back to the presets.
- solve: optional SolveDirective (see Acreage Solver).
- return_dataframes: when true, the response also carries the row-level tables.

Response (ScenarioResponse):

- inputs: the resolved inputs after defaults were applied, including species_codes and the merged financial_params with rates as fractions. When a solve ran, net_acres is the solved value.
- summaries: one ScenarioSummary per protocol: Protocol, net_acres, total_net, npv_yr, npv_year, npv_per_acre.
- model_source: fvs or coefficients.
- proforma_rows, carbon_rows, cu_rows: present only when return_dataframes is true.

Errors: a KeyError or ValueError inside the pipeline, including unreachable solve targets, returns 400 with the message in detail.

POST /scenario/bulk

Evaluates up to 1000 ScenarioRequest objects in one request. The response has results and errors; results[i] is null exactly when errors contains an entry with index i. Each error carries the same message that /scenario/run would have returned. Scenarios are evaluated in parallel when BULK_WORKERS is above 1; BULK_WORKER_MODE selects thread or process pools. The dashboard's sensitivity grid uses this endpoint.

POST /scenario/solve-tpa

Sweeps planting density and returns the range where NPV meets a target. See TPA Sweep.

Acreage Solver

The solve directive on /scenario/run asks the service to find net_acres instead of taking it as input.

SolveDirective fields:

- variable: must be net_acres.
- target: tnr (Total Net Revenue) or npv. Defaults to tnr.
- value: the target amount in dollars.

How it works. For a fixed protocol, fixed financial parameters, and fixed NPV horizon, both TNR and NPV are exactly linear in net_acres: revenue and per-acre costs scale with acreage, and the fixed costs do not. The service runs the proforma at 1 acre and at 10 acres, recovers the slope and intercept, and solves the line for the target. The result is then run through the normal pipeline so the returned summaries are the verified figures at that acreage.

Constraints:

- Exactly one protocol. A solve with more than one protocol returns 400.
- The slope must be nonzero. If the scenario produces no credits, the metric does not change with acreage and the solve fails with 400.
- The solved acreage must be positive. A target that is unreachable with the given inputs, for example an NPV target below the fixed-cost floor, returns 400.

The dashboard's breakeven acreage uses target npv with value 0 by default; the user can set any NPV target.

TPA Sweep

NPV is not linear in planting density. Too few trees produce little carbon; too many trees raise planting cost and increase competition and mortality in the growth model. The relationship is a hump, so there is no closed-form inverse. POST /scenario/solve-tpa grid-evaluates NPV across a range of densities and reads the feasible range off the curve, recovering both edges.

Request (TpaSweepRequest) extends ScenarioRequest with:

- mode: scalar, species, or per_species. Defaults to scalar.
    - scalar: multiplies the whole species mix by a factor k. The swept variable is k, with k equal to 1 at the base mix.
    - species: varies one species' TPA while holding the others fixed. The swept variable is that species' TPA.
    - per_species: runs the species sweep for every species in the mix and returns one result per species.
- species: index or species code of the species to vary. Required for mode species.
- target_npv: the total-NPV hurdle, summed across protocols. Required.
- op: >=, <=, or ==. Defaults to >=.
- grid: optional TpaGrid. lo_factor and hi_factor bound the sweep relative to the base value (defaults 0.25 and 4.0). lo and hi are absolute bounds and take precedence. max_value is a hard ceiling that clamps hi, which the dashboard sets to the model's TPA cap. steps is the number of grid points (default 25).
- include_curve: whether to return every evaluated point. Defaults to true.

Response (TpaSweepResponse):

- mode, op, target_npv, metric (always total_npv).
- base_species_tpa and species_codes.
- results: one TpaRangeResult per swept dimension.

Each TpaRangeResult has:

- species_index and species_code (null for scalar mode).
- variable: k or tpa.
- intervals: the maximal runs of the grid where the condition holds. Interval edges are linearly interpolated between adjacent grid points. For op == each crossing of the target is a zero-width interval.

- range: the outer bounding box over all intervals, or null if nothing on the grid satisfies the condition.
- lo_clipped and hi_clipped: true when the feasible region runs off the edge of the swept grid, meaning the true edge lies beyond it. The dashboard reports a clipped low edge as "profitable from the minimum density" and a clipped high edge as "non-binding up to the cap".
- curve: the evaluated points, each with x, npv, and the full species_tpa at that point.

Each grid point is a full scenario evaluation, so a 25-step sweep costs 25 scenario runs. Per-species mode multiplies that by the number of species.

Dashboard Solver Page

The Solver lives in the Project Builder under Site Selection and Planting Design. It requires a site to be selected on the map. Its inputs are:

- FVS variant (when the location has several sub-variants) and PCT level, read from the registry.
- Survival, site index, and species TPA, seeded from the variant preset and independent of the Planting Design values.
- Protocol: a single protocol, since solves run one protocol at a time.
- Target NPV and NPV horizon.
- Financial assumptions, grouped as Financial Inputs, Implementation Costs, and Registry-related Fees. They are seeded from the selected protocol's preset on first render, after which edits persist.

The page offers two breakeven variables:

Breakeven acreage. Calls POST /scenario/run with a solve directive for target npv. The headline shows Breakeven Net Acres, NPV at the horizon, and Total Net Revenue, rounded to the nearest hundred. An Apply button copies the solved scenario into Planting Design.

Sensitivity grid. The user picks one or two levers to vary. Available levers are discount rate (percent), ERT price, issuance fee per ERT, and NPV horizon. Each lever has preset values and, except for the horizon, accepts typed custom values. The page builds one solve scenario per combination with build_grid_scenarios and sends them in a single POST /scenario/bulk call, capped at the server's 1000-scenario limit. Results render as a heatmap and table of breakeven acres; cells whose solve failed show the API's error.

Breakeven TPA. Calls POST /scenario/solve-tpa. The user picks the mode: Scalar varies the whole mix and reports a multiplier and the equivalent per-species densities; Single species solves one species; Per species reports a range for each species. The grid's upper bound is clamped to the model's TPA cap. Results show the breakeven density, the NPV curve across the sweep, and an Apply button where a single density was found.

The builders in utils/functions/solver.py are Streamlit-free and are the reference for assembling payloads by hand:

- build_solve_scenario: a /scenario/run payload with a solve directive. Takes rates in percent and converts them.
- build_grid_scenarios: the cross product of swept lever values into solve scenarios, returning the scenarios and the lever values that produced each.
- build_tpa_breakeven_scenario: a /scenario/solve-tpa payload.

Planting Design Financials

The Project Financials section of Planting Design uses the same endpoints without solving. It calls POST /carbon/calculate for the carbon projection, POST /carbon/units for the selected protocol, and POST /proforma/compute with the editable inputs on the left and the protocol's fixed assumptions on the right. It

shows the yearly proforma, the summary metrics, and the cumulative net revenue chart with discounted and undiscounted series. The Solver's Apply buttons prefill this section.

Python Client

aff_dash_client wraps the scenario endpoints:

- AFFDashClient(api_base_url=None, api_key=None, timeout=60): defaults to the deployed dev API and the CARBON_API_BASE_URL and CARBON_API_KEY environment variables. Pass local_models_dir to run in-process against local model files instead of the API.
- defaults(variant, loccode): returns a Defaults object with with_overrides(**kwargs) for building a modified scenario.
- run(...): a single /scenario/run; returns a ScenarioResult with summaries per protocol and, when return_dataframes is set, the proforma, carbon, and carbon-unit tables as DataFrames.
- solve_for_tnr(...) and solve_for_npv(...): /scenario/run with a solve directive. They return a ScenarioResult whose acreage property is the solved net_acres and whose tnr and npv properties map each protocol to its value.
- solve_tpa_range(...): /scenario/solve-tpa; returns a TpaSweepResult.
- run_bulk(scenarios) and run_many(...): /scenario/bulk.

The scenario command-line tool (uv run scenario --variant PN --loccode 609 --net-acres 500) calls the same client. With --target-tnr or --target-npv it solves for acreage.

Example:

```
None
from aff_dash_client import AFFDashClient

client = AFFDashClient(api_key="<key>")
d = client.defaults(variant="PN", loccode="609")
result = client.solve_for_npv(
    variant="PN", loccode="609", target_npv=0, npv_year=40,
    financial_params={"ACR": {"price_per_ert_initial": 35.0}},
)
print(result.acreage, result.tnr["ACR"], result.npv["ACR"])
```

Example with curl:

```
None
curl -X POST "$CARBON_API_BASE_URL/scenario/run" \
  -H "Authorization: Bearer <key>" -H "Content-Type: application/json" \
  -d '{"variant": "PN", "loccode": "609", "protocols": ["ACR"],
       "npv_year": 40,
       "solve": {"variable": "net_acres", "target": "npv", "value": 0}}'
```

Errors and Limits

- 400: a validation or pipeline error, including unreachable solve targets, more than one protocol with a solve directive, an unknown species for mode species, an invalid mode or op, or a bulk request that is empty or larger than 1000.
- 401: missing or invalid API key. 403: a client-role key calling an admin route.
- 404: /scenario/defaults for a variant and location with no registered model.
- /scenario/bulk accepts up to 1000 scenarios. The dashboard's grid respects the same cap.

- The API's request timeout on Cloud Run is 3600 seconds. Large per-species sweeps and large bulk batches are the slow calls.

Operational Notes

- Financial defaults ship in conf/base/proforma_presets.json and are synchronized into the model store at startup. Model Management can edit them outside production.
- Rates in requests may be percent or fraction; rates in responses are fractions.
- The NPV horizon defaults to 40 years on the scenario endpoints and 20 years on /proforma/compute when the caller does not set it.
- The solver runs one protocol at a time. Compare protocols by issuing one solve per protocol, or one bulk request containing all of them.